

Project Document

Memory-Driven Conversational AI (Mini ChatGPT)

1. Project Overview

Project Name: Memory-Driven Mini ChatGPT

Type: AI Chatbot with Memory

Simple Idea:

We are building a chatbot like ChatGPT that can: - Remember what user said before - Understand conversation context - Give personalized replies - Improve over time

2. Objectives (Simple)

- Make chatbot that talks like human
 - Remember recent messages (short memory)
 - Remember important user info (long memory)
 - Give better answers using past data
 - Prepare system for future features (like tools & agents)
-

3. Core Concepts (Easy Explanation)

♦ Short-Term Memory (STM)

👉 What it means: - Last 5-10 messages - Helps chatbot continue conversation

👉 Example: User: My name is Anshul
Bot remembers it for next few messages

♦ Long-Term Memory (LTM)

👉 What it means: - Stores important things about user - Like preferences, interests, goals

👉 Example: User: I like gaming laptops
Bot stores this and uses later

♦ Memory Retrieval

👉 What it does: - Finds useful past data - Sends it to AI before answering

👉 Simple: "Search old memory → pick relevant → use in answer"

🧩 4. System Architecture (Simple Flow)

```
graph TD; A[User Message] --> B[Check Recent Chat (Short Memory)]; B --> C[Search Old Memory (Long Memory)]; C --> D[Send both to AI Model]; D --> E[Generate Response]; E --> F[Save important info];
```

⚙️ 5. Key Features (Easy)

✅ Understand Context

Bot remembers recent chat

✅ Personal Replies

Bot uses user preferences

✅ Smart Memory

Only important data is saved

✅ Works Across Sessions

Bot remembers even after refresh

🔥 6. Advanced Features (Important)

💣 Memory Summary

Long chat → short summary → store

💣 Forget Option

User can say: "Don't remember this"

Multi User

Each user has separate memory

Security

Avoid bad/malicious inputs

7. Prompt Design (Simple)

You are a helpful AI assistant.

User Info:

{long_term_memory}

Recent Chat:

{short_term_memory}

User Question:

{query}

8. Tech Stack (Simple)

Backend

- FastAPI

AI Layer

- LangChain
- LangGraph (future)
- OpenAI API

Database

- PostgreSQL (chat history)
- FAISS / ChromaDB (memory)

Frontend

- React / Next.js
-

9. Team Roles (Very Clear)

Member 1 – Memory System

- Store and fetch memory
- Vector DB setup

Member 2 – AI Logic

- Prompt design
- Response generation

Member 3 – Backend

- APIs
- Database connection

Member 4 – Frontend

- Chat UI
 - User experience
-

10. Workflow (Step-by-Step)

1. User sends message
 2. Get recent messages
 3. Get relevant past memory
 4. Send all to AI
 5. Generate reply
 6. Save useful info
-

11. Future Scope

Later we can add: - Web search tool - Image generation - File upload analysis - Full AI agents

12. Why This Project is Strong

- Real industry system
 - Uses modern AI concepts
 - Covers:
 - NLP
 - RAG
 - Memory system
 - Easy to extend in future
-

13. Final Outcome

Our chatbot will: - Remember user - Understand context - Give smart replies - Improve over time

Conclusion

This is not just a chatbot.

It is a **smart AI system that learns about the user.**

End Goal:

Build AI that *remembers + understands + improves*
